

QMC Summer School 2025

Relaxation of atomic structures with STALK

22 July 2025

Juha Tiihonen, juha.tiihonen@tuni.fi



Tampere University, Finland

Funding: U.S. Department of Energy, Office of Science, Basic Energy Sciences, Materials Sciences and Engineering Division, as part of the Computational Materials Sciences Program and Center for Predictive Simulation of Functional Materials.





How to stalk?

1. Locate prey
2. Follow their moves
3. Keep the noise down

Outline

See QMCPACK workshop 2023 lecture by Jaron T. Krogel for more background



Surrogate Hessian method for geometry optimization, Jaron Krogel, QMCPACK Workshop 2023, 12/18



Background: On geometry relaxation

Theory: Surrogate Hessian
accelerated parallel Line-search
algorithm

Practice: Surrogate Theory
Accelerated Line-search Kit (STALK)

Geometries from first principles

The philosophy of first principles: predictions self-consistently from theory/fundamentals, with no empirical parameters

Equation of state (at 0 K, Born–Oppenheimer):

$$\hat{H}(R)\Psi(R) = E_0(R)\Psi(R) \quad R \equiv \{\mathbf{d}_1, \mathbf{d}_2, \dots \mathbf{d}_N\} \quad \text{Ionic coordinates}$$

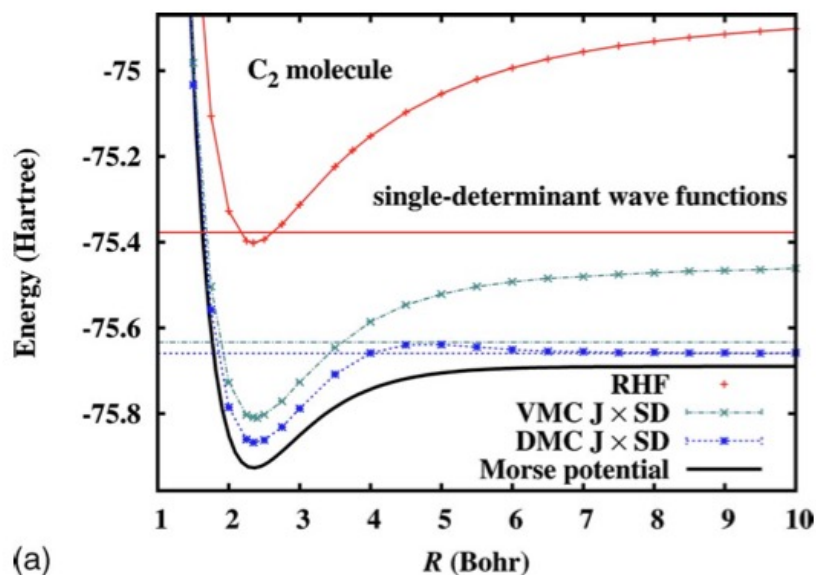
$$\hat{H}(R) = -\frac{1}{2} \sum_i \nabla_i^2 - \sum_{\alpha} \sum_i \frac{Z_{\alpha}}{|\mathbf{r}_i - \mathbf{d}_{\alpha}|} - \sum_i \sum_{i < j} \frac{Z_{\alpha}}{|\mathbf{r}_i - \mathbf{r}_j|}$$

The predicted **equilibrium atomic structure** is the energy-minimizing geometry R^*

$$R^* = \operatorname{argmin}_R [E_0(R)]$$

Potential energy surface (PES)

$E(R)$: Potential energy surface



Toulouse & Umrigar JCP **128** 174101 (2008)

Each PES is different:

$$E_{RHF}(R) \neq E_{VMC}(R) \neq E_{DMC}(R) \neq E_{EXPT}(R)$$

Each predicted geometry is different:

$$R_{RHF}^* \neq R_{VMC}^* \neq R_{DMC}^* \neq R_{EXPT}^*$$

Also within DMC:

$$R_{DMC@RHF}^* \neq R_{DMC@DFT}^* \neq R_{DMC-AREP}^* \text{ etc}$$

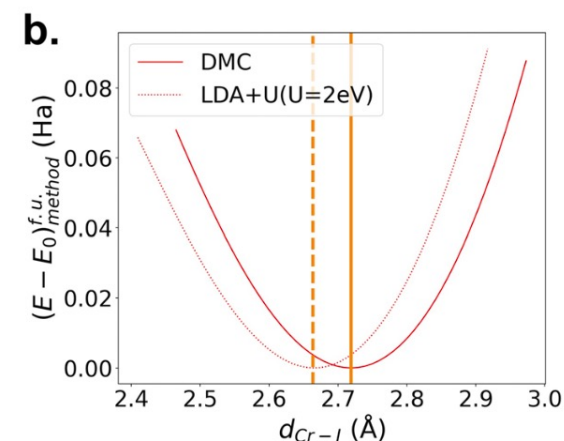
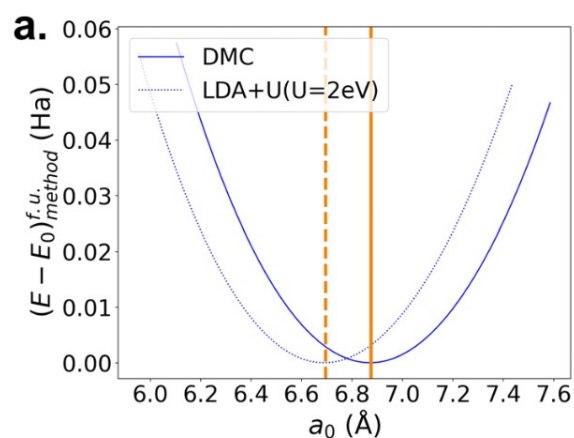
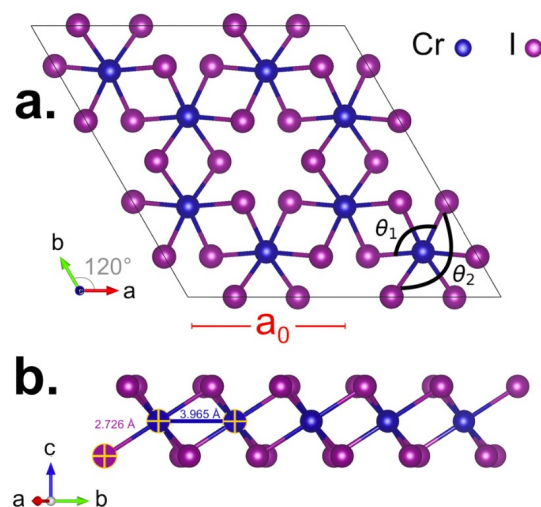
Fully first-principles simulation should use self-consistent geometries (but often doesn't):

$$O_{DMC}(R_{EXPT}^*) \neq O_{DMC}(R_{DFT}^*) \neq O_{DMC}(R_{DMC}^*)$$

[See e.g. Hyeondeok Shin: Real-world calculations: bulk and 2D monolayer GeSe](#)

Example: CrI_3 monolayer

Staros et al. **JCP** 156, 014707 (2022)



$$E(p) = E(p^*) + \frac{1}{2}\lambda_p(p - p^*)^2 + O((p - p^*)^3)$$

The PES minima differ ($\sim 3\%$) between DFT and DMC:

$$p_{LDA+U}^* \neq p_{DMC}^*$$

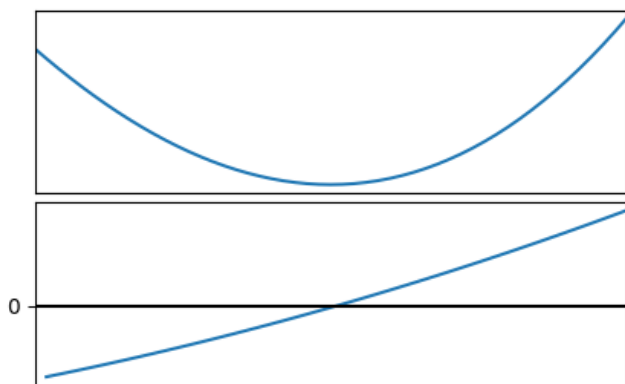
DFT holds transferable information of the PES:

$$\lambda_p(LDA + U) \approx \lambda_p(DMC)$$

The challenge of a (noisy) minimization

$$R^* = \operatorname{argmin}_R [E_0(R)]$$

Smooth PES:



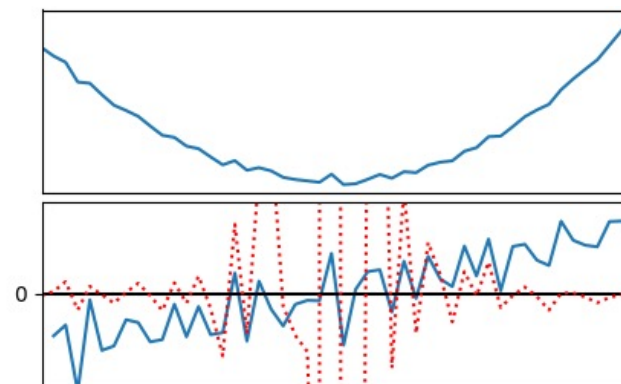
Energy

Force

Easily optimized by minimizing forces/gradients

- Push-button approach for most codes/methods

Noisy PES:



Minimization of discontinuous, noisy PESs is complicated

- **QMC force estimators** face challenges with bias, computational cost, signal-to-noise ratio
- **PES is the most robust QMC observable**

Surrogate line-search: overview

Surrogate Hessian accelerated structural optimization for stochastic electronic structure theories

Cite as: J. Chem. Phys. 156, 054104 (2022); doi: 10.1063/5.0079046

Submitted: 17 November 2021 • Accepted: 6 January 2022 •

Published Online: 2 February 2022

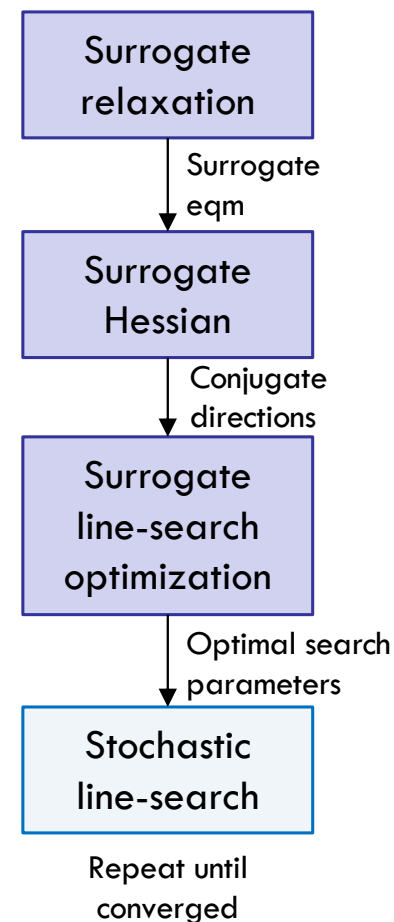


Juha Tiihonen,¹ Paul R. C. Kent,² and Jaron T. Krogel^{1,a)}

An energy-based relaxation algorithm for stochastic theories

- **Practical:** performance over elegance
- **Any stochastic theory (with a reasonable surrogate):**
 - VMC, DMC, AFQMC, CIPSI, noisy intermediate-scale quantum computer (NISQ)

arXiv: 2404.02951



Optimizing the optimization

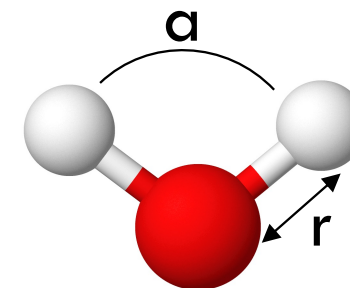
1. What is the least amount of information needed to represent R_{DMC}^* ?
 2. What is the most efficient way to crawl $E_{DMC}(R)$ towards minimum?
 3. How to ensure convergence?
 4. How to control systematic and statistical accuracy of the result?
 5. How to control statistical/computational cost of the workflow?
 6. How to exploit parallelism and supercomputing facilities?
- **Many new concepts coming up!**

Line-search minimization of a PES

$$R^* = \operatorname{argmin}_R [E_0(R)] \rightarrow \mathbf{p}^* = \operatorname{argmin}_{\mathbf{p}} [E_0(\mathbf{p})]$$

1) Mapping to irreducible parametric subspace: $R \rightarrow \mathbf{p}$

- Eliminates rotation, translation & other symmetries
- Reduces degrees of freedom

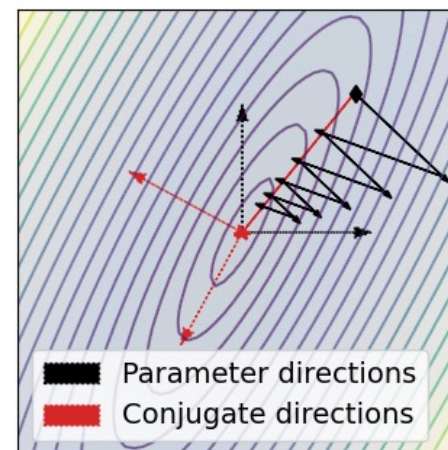


From $3N = 9 \rightarrow 2$ degrees of freedom

2-3) Hessian expansion of the PES:

$$\begin{aligned} E(\mathbf{p}) &\approx E_0 + \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^T H_{\mathbf{p}} (\mathbf{p} - \mathbf{p}^*) \\ &= E_0 + \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^T U^T \Lambda U (\mathbf{p} - \mathbf{p}^*) \\ &= E_0 + \frac{1}{2} \sum_i \lambda_i x_i^2 \quad \rightarrow x_i = U_i(\mathbf{p} - \mathbf{p}^*) \end{aligned}$$

Columns of eigenmatrix U are **conjugate directions** that allow optimal convergence to the minimum via independent line-searches



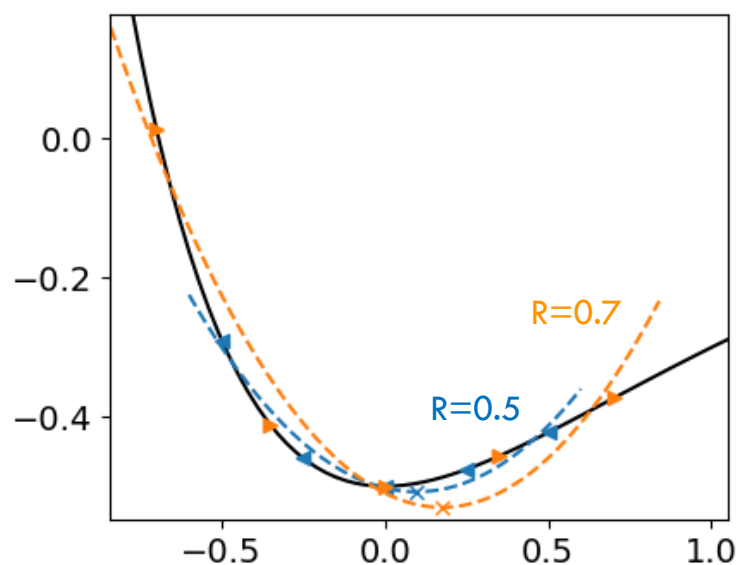
Oscillatory convergence

Immediate convergence

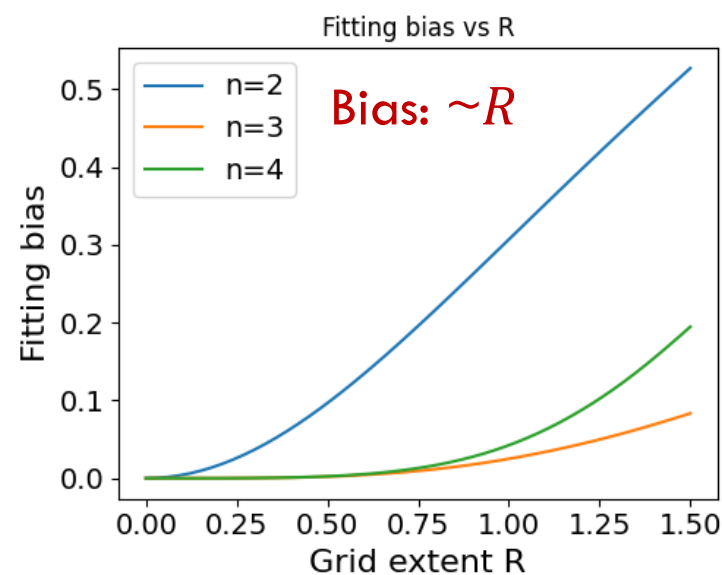
Try it out!

Line-search minimization: fitting bias

Harmonic ($n=2$) fit to $M=5$ points
along a Morse potential



Fitting bias ensues with the grid extending far away from eqm.

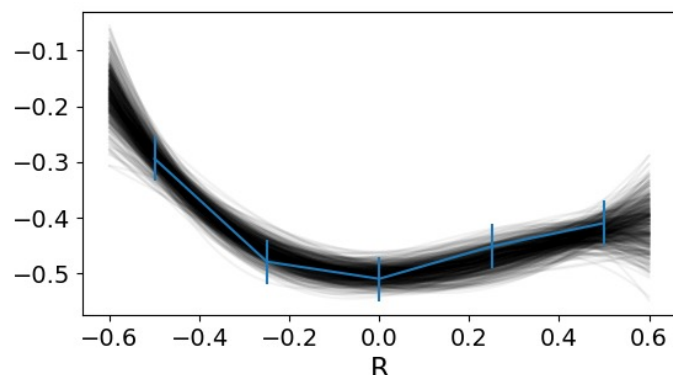


Higher-order fits ($n > 2$) get lower bias

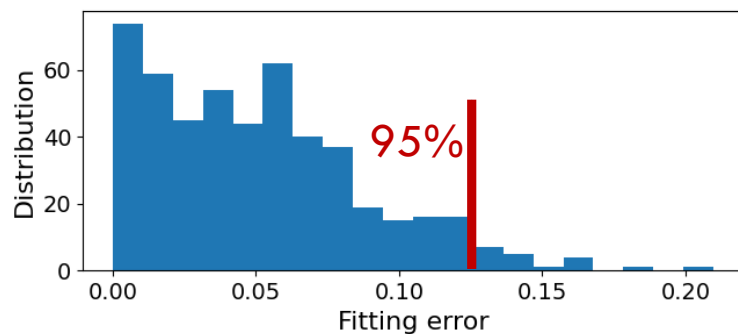
- For Morse, $n=3$ is remarkably better than $n=4$

Line-search minimization: noise

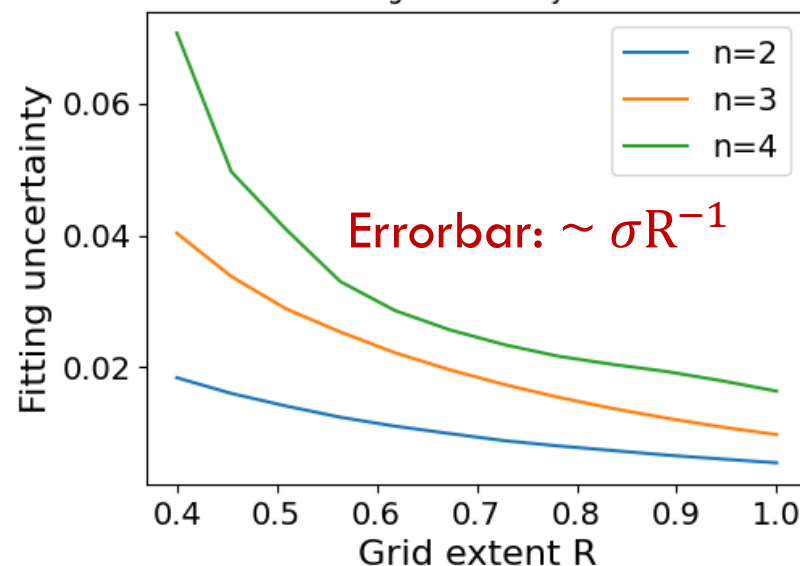
Finite errorbars of the PES ($\sigma > 0$) induce errorbars for the predicted minimum



Fitting uncertainty (95% confidence) can be estimated by statistical resampling



Fitting uncertainty vs R



- For given input noise, the fitting uncertainty fades with higher R (better signal-to-noise ratio)
- Higher-order fits ($n > 2$) tolerate lower noise

Total error tolerance & optimization

Total error = bias + errorbar < tolerance

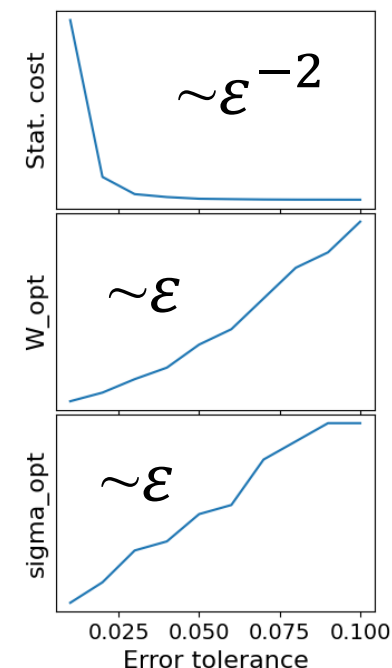
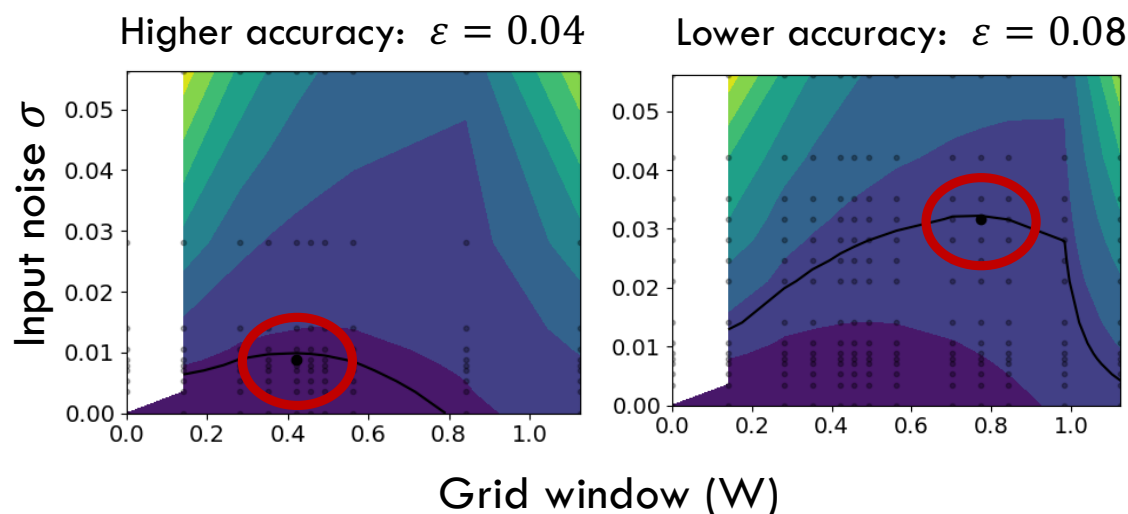
“Window”: $W = \frac{\lambda R^2}{2}$

Bias: $\sim W^{>0}$

Errorbar: $\sim \sigma W^{-0.5}$

Statistical cost: $\sim \sigma^{-2}$

Error tolerance: ε



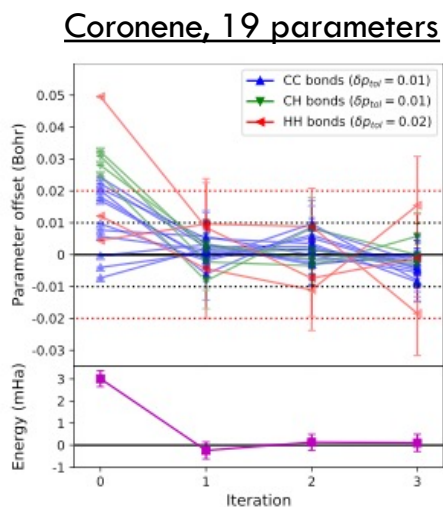
4-5) Meet a tolerance while minimizing statistical cost:

- Find W_{opt}, σ_{opt} that maximize σ such that $\text{Error}(W, \sigma) \leq \varepsilon$
- Interplay between P dimensions is non-trivial and must be treated carefully/numerically

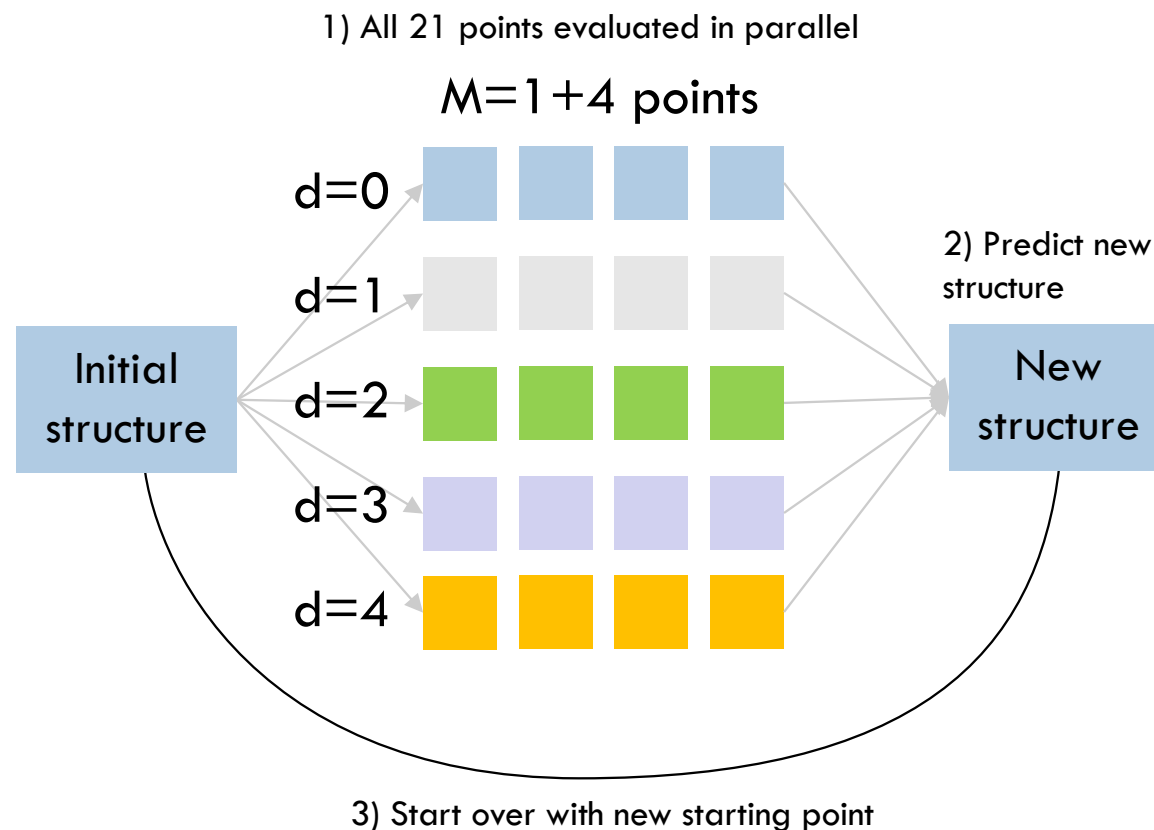
Parallel line-search iteration

6) Parallel line-search

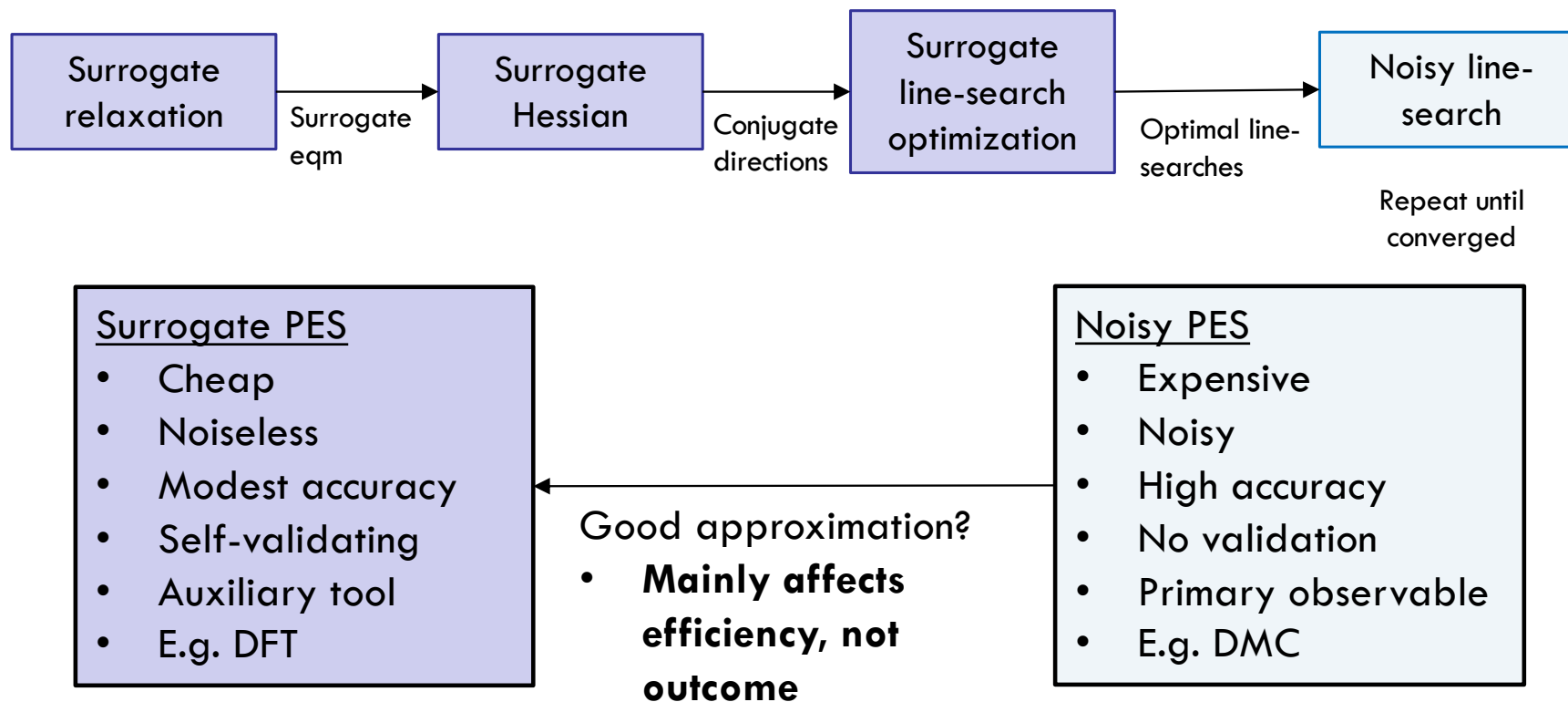
- All directions can be searched at once
- Ideally converges in a single parallel iteration
- Stopping criterion: saturation



J. Chem. Phys. **156**, 054104 (2022)



Complete workflow and surrogacy





How to ~~stalk~~? STALK

1. Locate prey
 3. Locate noisy energy minimum
2. Follow their moves
 2. Follow conjugate directions
3. Keep the noise down
 1. Use a noiseless surrogate

Surrogate theory accelerated line-search kit (STALK)

A Python implementation of the surrogate accelerated line-search method for noisy parameter optimization.

<https://github.com/QMCPACK/stalk>

Installation:

```
git clone https://github.com/QMCPACK/stalk.git stalk
export PYTHONPATH=$PYTHONPATH:"$PWD/stalk"
```

```
pip3 install stalk_qmc
```

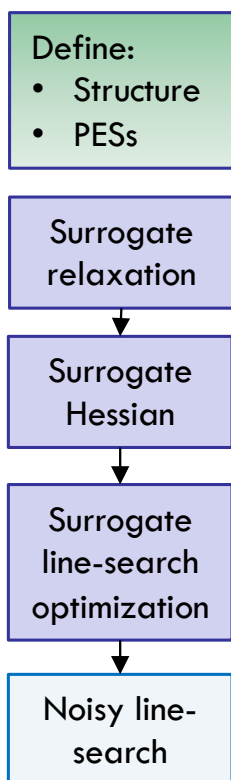
Documentation:

<https://stalk.readthedocs.io/en/stable/>

Features:

- Nexus integration
 - QMCPACK
 - Quantum Espresso
 - PySCF
- Easily customized
- Unit tested
- In development

STALK project overview



Recommended input file layout:

- `pseudos/*.{upf,xml}`
 - Contains pseudopotentials for Nexus
- `params.py`
 - Parametric mappings
 - PES input recipes
- `run0_relax.py`
 - Relaxation of an initial structure
- `run1_hessian.py`
 - Calculation of the parametric Hessian
- `run2_surrogate.py`
 - Creation and optimization of the surrogate
- `run3_ls.py`
 - The actual line-search iteration loop
- Easy to dissect, analyze and branch different parts

Alternative implementations:

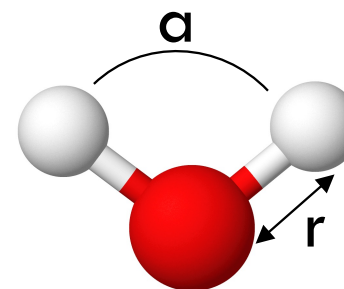
- Python CLI
- The whole project in one script
- Jupyter notebook

Typical output file layout

- `relax/*`
- `hessian/*`
- `surrogate/*`
- `{dmc,srg}_ls/*`
- Clear intermediate results if something changes!

params.py: The parametric mappings

- Define the atomic structure to be optimized
- Define the parametric mappings
 - **Forward:** position $\rightarrow$ parameters
 - **Backward:** parameters $\rightarrow$ positions



params.py

```
# Natural forward mapping using bond lengths and angles
def forward_natural(pos: ndarray):
    pos = pos.reshape(-1, 3) # make sure of the shape
    # for easier comprehension, list particular atoms
    O0 = pos[0]
    H0 = pos[1]
    H1 = pos[2]
    # for redundancy, calculate mean bond lengths
    r = mean_distances([
        (O0, H0),
        (O0, H1)
    ])
    a = bond_angle(H0, O0, H1, units='rad')
    params = [r, a]
    return params
# end def
```

```
# Backward mapping: produce array of atomic positions
from bond length and angle
def backward_natural(params: ndarray):
    r, a = tuple(params)
    # Transform bond angle to triangular angle
    a = (pi - params[1]) / 2
    # place atoms on the xy-plane
    O0 = [0.0, 0.0, 0.0]
    H0 = [r * cos(a), r * sin(a), 0.0]
    H1 = [-r * cos(a), r * sin(a), 0.0]
    pos = array([O0, H0, H1])
    return pos
# end def
```

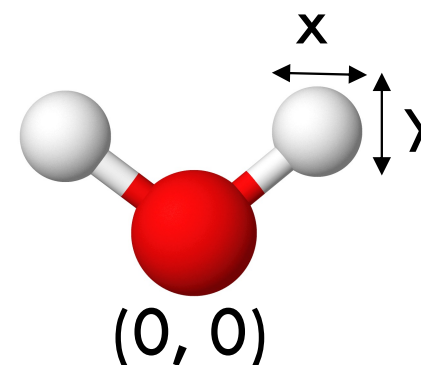
params.py: Alternative mapping

The same subspace can be mapped in different ways

- *In principle*, the choice does not matter
- In practice, some mappings are more practical/efficient
- This is an open research problem

params.py

```
# Auxiliary forward mapping using Cartesian support variables
def forward(pos: ndarray):
    pos = pos.reshape(-1, 3) # make sure of the shape
    # for easier comprehension, list particular atoms
    O0 = pos[0]
    H0 = pos[1]
    H1 = pos[2]
    # Parameter #0, the horizontal offset
    x = mean_param([
        (H0 - O0)[0], # first H on the 'right'
        -(H1 - O0)[0] # second H on the 'left'
    ])
    y = mean_param([
        (H0 - O0)[1], # positive value when H's are 'above' O
        (H1 - O0)[1]
    ])
    return array([x, y])
# end def
```



```
# Backward mapping: produce array of atomic
positions from x, y offsets
def backward(params: ndarray):
    x, y = tuple(params)
    O0 = [0.0, 0.0, 0.0]
    H0 = [x, y, 0.0] # first H on the 'right'
    H1 = [-x, y, 0.0] # second H on the 'left'
    pos = array([O0, H0, H1])
    return pos
# end def
```

STALK project definition: The PES

params.py

PES job: parameter set -> structure -> energy

```
def pes_pyscf(structure: ParameterStructure, xc='pbe', **kwargs):
    print(f'Computing: {structure.label} ({xc})')
    mf = kernel_pyscf(structure=structure)
    mf.xc = xc
    e_scf = mf.kernel()
    return e_scf, 0.0
# end def

# Allow easy variations of the theoretical PES
pes_pbe = PesFunction(pes_pyscf, {'xc': 'pbe'})
pes_b3lyp = PesFunction(pes_pyscf, {'xc': 'b3lyp'})
```

Relaxation job: initial structure -> relaxed structure

```
def relax_pyscf(structure: ParameterStructure, outfile='relax.xyz', xc='pbe'):
    mf = kernel_pyscf(structure=structure)
    mf.xc = xc
    mf.kernel()
    mol_eq = optimize(mf, maxsteps=100)
    # Write to external file
    tofile(mol_eq, outfile, format='xyz')
# end def
```

Common kernel ensures relax/PES consistency

```
def kernel_pyscf(structure: ParameterStructure):
    atom = []
    for el, pos in zip(structure.elem, structure.pos):
        atom.append([el, tuple(pos)])
    # end for
    mol = gto.Mole()
    mol.atom = atom
    mol.verbose = 3
    mol.basis = 'ccpvdz'
    mol.unit = 'A'
    mol.ecp = 'ccecp'
    mol.charge = 0
    mol.spin = 0
    mol.symmetry = False
    mol.cart = True
    mol.build()
    mf = dft.RKS(mol)
    return mf
# end def
```

STALK project definition: Nexus PES

params.py

NOTE! In 'nexus mode', the PES function generates jobs and is wrapped with a **loader**

```
pes_pyscf = NexusPes(
    pes_pyscf_job,
    # pes_pyscf_job outputs energy in energy.dat
    loader=FilesLoader({'suffix': 'energy.dat'})
)
```

```
pes_dmc = NexusPes(
    dmc_pes_job,
    loader=QmcPes({'suffix': '/dmc/dmc.in.xml', 'qmc_idx': 1})
)
```

```
# Nexus generator for DMC PES workflow
def dmc_pes_job(
    structure: Structure,
    **kwargs
):
    system = generate_physical_system(
        # Physical system inputs
    )
    # Generate orbitals with PySCF
    scf = generate_pyscf(
        # PySCF inputs
    )
    c4q = generate_convert4qmc(
        # Convert4qmcpack inputs
        dependencies=(scf, 'orbitals'),
    )
    # Optimize J1 + J2
    opt = generate_qmcpack(
        # QMCPACK inputs for optimization
        dependencies=[(c4q, 'orbitals')],
    )
    # Generate DMC job
    dmc = generate_qmcpack(
        # QMCPACK inputs for DMC
        dependencies=[(c4q, 'orbitals'), (opt, 'jastrow')],
    )
    return [scf, c4q, opt, dmc]
# end def
```

STALK project, #0: Surrogate relaxation

- Init parameterized structure
- Obtain the energy-minimizing geometry using the relaxation job

run0_relax.py

```
from params import forward, backward, backward_natural, relax_pyscf

params_init_natural = array([0.97, 104.0 / 180 * pi])
pos_init = backward_natural(params_init_natural)
structure_natural = ParameterStructure(
    forward=forward,
    backward=backward,
    pos=pos_init,
    elem=['O'] + 2 * ['H'],
    units='A',
    label='init'
)

outfile = 'relax.xyz'
relax_pyscf(structure_natural, outfile)
geom = XyzGeometry({'suffix': outfile}).load('./')
new_params = structure_natural.map_forward(geom.get_pos())
structure_relax = structure_natural.copy(params=new_params, label='eqm')
print(structure_natural)
print(structure_relax)
```

```
ParameterStructure (init)
params:
  p0          0.7644
  p1          0.5972
consistent: yes
pos (3 atoms)
  O    0.0000    0.0000    0.0000
  H    0.7644    0.5972    0.0000
  H   -0.7644    0.5972    0.0000
periodic: no
ParameterStructure (eqm)
params:
  p0          0.7611
  p1          0.6015
consistent: yes
pos (3 atoms)
  O    0.0000    0.0000    0.0000
  H    0.7611    0.6015    0.0000
  H   -0.7611    0.6015    0.0000
periodic: no
```

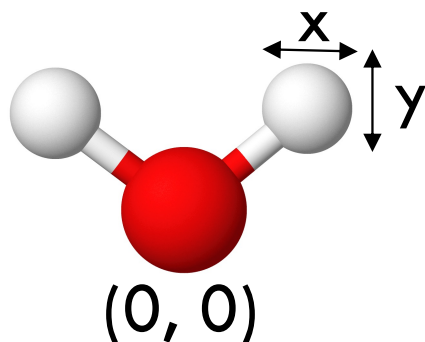
STALK project #1: Surrogate Hessian

- Calculate the parametric subspace Hessian using finite-difference method

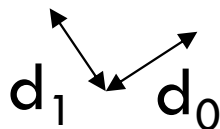
run1_hessian.py

```
from params import pes_pbe
from run0_relax import structure_relax

hessian = ParameterHessian(structure=structure_relax)
hessian_file = 'hessian.dat'
hessian.compute_fdiff(pes=pes_pbe)
savetxt(hessian_file, hessian.hessian)
print('Hessian:')
print(hessian)
```



Conjugate directions in (x, y) basis



```
Computing Hessian with finite-difference method:
Computing: eqm (pbe)
Computing: eqm_p0+0.01 (pbe)
Computing: eqm_p0-0.01 (pbe)
Computing: eqm_p0+0.01_p1+0.01 (pbe)
Computing: eqm_p0+0.01_p1-0.01 (pbe)
Computing: eqm_p0-0.01_p1+0.01 (pbe)
Computing: eqm_p0-0.01_p1-0.01 (pbe)
Computing: eqm_p1+0.01 (pbe)
Computing: eqm_p1-0.01 (pbe)
Hessian:
ParameterHessian
  hessian:
    2.756884 1.375956
    1.375956 1.567770
  Conjugate directions:
    Lambda    Direction
    3.661244  +0.835661 +0.549246
    0.663410  -0.549246 +0.835661
```

STALK project #2: The surrogate 1/2

Construct a *surrogate* of the parallel line-search along conjugate directions

run2_surrogate.py

```
from params import pes_pbe
from run1_hessian import hessian

surrogate_file = 'surrogate.p'
surrogate = TargetParallelLineSearch(
    path='surrogate/',
    fit_kind='pf3',
    load=surrogate_file,
    structure=hessian.structure,
    hessian=hessian,
    pes=pes_pbe,
    window_frac=0.5,
    M=15
)
# Eliminate relaxation deficiencies by
bracketing true minimum
surrogate.bracket_target_biases()
```

Sample the directions generously ($M=15$)
to characterize bias

- $W_{\max}$ defined by window_frac

run2_surrogate.py

```
epsilon_p = [0.01, 0.01]
surrogate.optimize(
    epsilon_p=epsilon_p,
    fit_kind='pf3',
    M=7,
    N=400,
    reoptimize=False,
    write=surrogate_file,
)
```

Choose parameter accuracy targets and
optimize for statistical cost

- Must specify fit kind and number of points M
- Involves blind guessing, as the practical connection of $W_{\max}$ and ϵ_p is not known *a priori*

STALK project #2: The surrogate 2/2

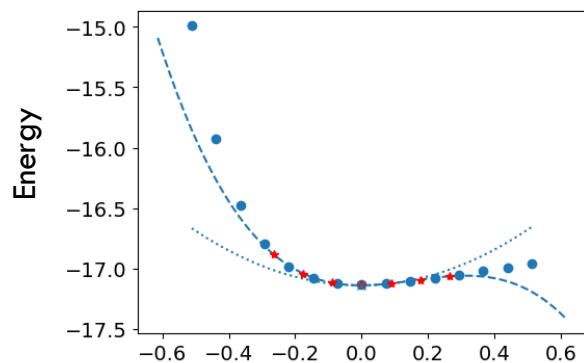
run2_surrogate.py

```
print(surrogate.sigma_opt)
print(surrogate.W_opt)
surrogate.plot()
surrogate.plot_error_surfaces()
plt.show()
```

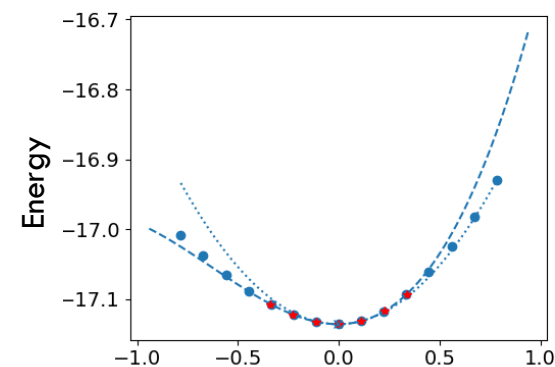
Blue dots: surrogate energies
 Dashed line: surrogate fit (poor)
 Dotted line: Hessian expansion
 Red stars: optimized grid

```
Optimizing to epsilon_p using line-search
p= 0 0.0200
p= 1 0.0200
Optimizing to epsilon_d
tls= 0 (epsilon= 0.0121 )
tls= 1 (epsilon= 0.0200 )
Optimization complete:
    d  target    error  rel.
    0  0.0121   0.0113 93.196%
    1  0.0200   0.0189 94.678%
    p  target    error  rel.
    0  0.0200   0.0120 60.033%
    1  0.0200   0.0181 90.588%
W_opt: [0.0822, 0.0286]
sigma_opt: [0.00112, 0.000556]
```

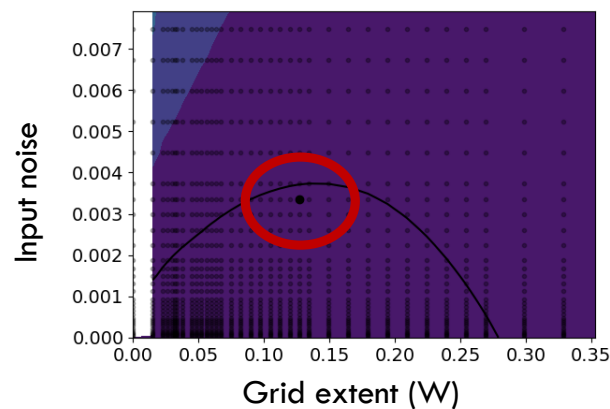
Direction #0: PES & error surface



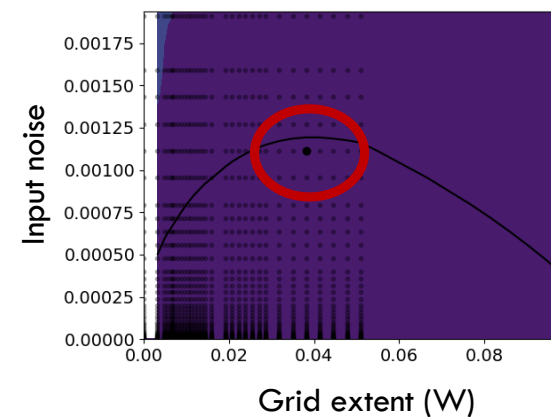
Direction #1: PES & error surface



d_0 displacement



d_1 displacement



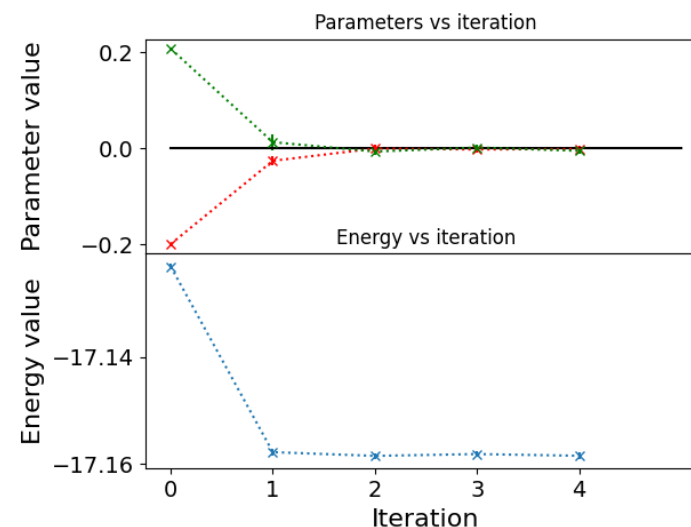
STALK project #3: Line-search iteration

run3_ls.py

```
from run0_relax_b3lyp import structure_relax as
structure_b3lyp
from params import pes_b3lyp
from run2_surrogate import surrogate

shifted_structure = surrogate.structure.copy()
shifted_structure.shift_params([-0.2, 0.2])
# Then generate line-search iteration object based on the
shifted surrogate
srg_ls = LineSearchIteration(
    surrogate=surrogate,
    structure=shifted_structure,
    path='b3lyp_ls',
    pes=pes_b3lyp,
)
# Iterate 4 times
for i in range(4):
    srg_ls.propagate(i, add_sigma=True)
# end for
# Evaluate the latest eqm structure
srg_ls.pls().evaluate_eqm(add_sigma=True)

print(srg_ls)
print('True (B3LYP) energy and params:')
print(structure_b3lyp.value, structure_b3lyp.params)
srg_ls.plot_convergence(targets=structure_b3lyp.params)
plt.show()
```



LineSearchIteration			
pls	Energy	p0	p1
0	-17.1231 +/- 0.0006	0.5611 +/- 0.0000	0.8015 +/- 0.0000
1	-17.1578 +/- 0.0006	0.7344 +/- 0.0097	0.6070 +/- 0.0164
2	-17.1585 +/- 0.0006	0.7599 +/- 0.0069	0.5874 +/- 0.0072
3	-17.1582 +/- 0.0006	0.7579 +/- 0.0059	0.5957 +/- 0.0070
4	-17.1585 +/- 0.0006	0.7590 +/- 0.0055	0.5890 +/- 0.0069

True (B3LYP) energy and params:
 -17.158202489715496 [0.75989 0.59367]

Troubleshooting and avoiding pitfalls

Stay mindful of

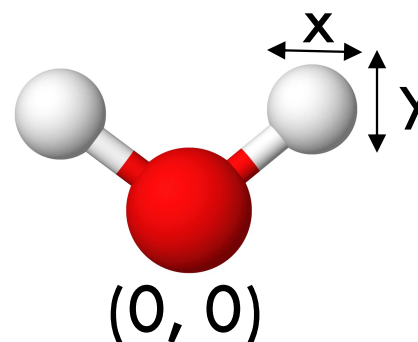
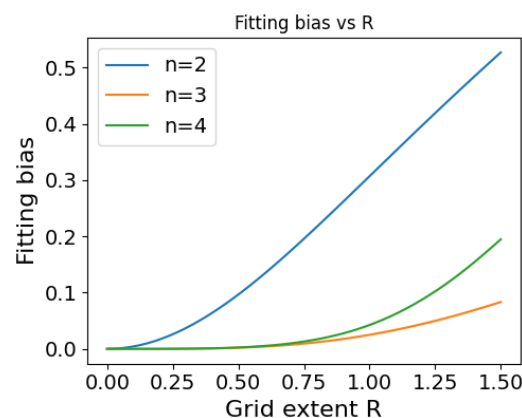
- The underlying theory / conventions
- Parametric mapping consistency
 - The forward/backwards mappings must match
- Unit conversions between codes
 - Quantum Espresso (Ry) vs QMCPACK (Ha)
- Reasonable tolerances
 - Too low error tolerances are unattainable
- Poor simulation data
 - Runs may fail, converge to wrong states, etc

Can't fix it?

- Report bugs!
- Ask for help!

Summer school examples

https://github.com/QMCPACK/qmc_summer_school_2025/tree/main/session7_stalk



0: Morse analysis

- Peek inside STALK and reproduce line-search analyses based on Morse potential
- <1 min on VM

1: H₂O on PySCF

- H₂O example on native PySCF
- Simulate stochastic PES with DFT + added white noise
- 1-5 mins on VM

2: H₂O on DMC

- Job management with Nexus
- Run actual DMC on QMCPACK
- 1-5 mins + 10-40 mins / DMC iter. on VM

Summary

- **Geometry relaxation** with QMC is a developing feature
 - Important for species with high structural sensitivity
 - Direct PES minimization (as opposed to forces) avoids estimator difficulties
- **Surrogate accelerations** boost algorithmic and statistical efficiency
 - Starting structure
 - Optimal search directions from Hessian
 - Error characterization for minimal statistical cost
- **STALK implements** an (almost) push-button PES minimization for DMC
 - Surrogate accelerations
 - Control over accuracy and cost
 - Nexus integration for QMCPACK setup, supercomputers and parallelism

